

Relazione per “shOOt”

Vera Murgia,
Kirolos Habashi,
Peter Ibrahim,
Leroy Fabbri

19 giugno 2026

Indice

Indice.....	2
Analisi.....	2
1.1 Descrizione e requisiti.....	2
Requisiti funzionali.....	3
Requisiti non funzionali.....	3
1.2 Modello del Dominio.....	3
Design.....	6
Architettura.....	6
Game state.....	7
Gerarchia delle entità.....	8
Design dettagliato.....	8
Vera Murgia.....	8
Peter Ibrahim.....	11
Kirolos Habashi.....	13
Leroy Fabbri.....	15
Sviluppo.....	16
Testing automatizzato.....	16
Vera Murgia.....	16
Peter Ibrahim.....	17
Kirolos Habashi.....	17
Leroy Fabbri.....	18
Note di sviluppo.....	18
Vera Murgia.....	18
Peter Ibrahim.....	19
Kirolos Habashi.....	20
Leroy Fabbri.....	21
Commenti finali.....	22
Autovalutazione e lavori futuri.....	22
Vera Murgia.....	22
Peter Ibrahim.....	23
Kirolos Habashi.....	23
Leroy Fabbri.....	24
Difficoltà incontrate e commenti per i docenti.....	24
Guida utente.....	25

Analisi

1.1 Descrizione e requisiti

ShOOt è un videogioco d'azione ispirato a Vampire Survivors¹ in cui il giocatore combatte in tempo reale contro dei nemici progressivamente più numerosi e forti. Il personaggio dispone di un'arma a proiettili, che può essere migliorata tramite upgrade ottenuti accumulando punti esperienza durante la partita.

La mappa di gioco non è generata casualmente, ma è predefinita; al suo interno si svolgono i combattimenti contro i nemici. Durante la partita, il giocatore ottiene punti esperienza eliminando i nemici, che permettono di migliorare progressivamente le proprie capacità. Accumulando esperienza, è possibile ottenere upgrade che rendono più agevole affrontare nemici sempre più forti.

Il gioco presenta una progressione interna alla partita, caratterizzata da una barra della vita e dalla presenza di boss, che rappresentano sfide più impegnative. Alla morte del personaggio, i progressi non vengono mantenuti: la partita termina e il giocatore può scegliere se ricominciare dall'inizio oppure uscire dal gioco.

Requisiti funzionali

- Il giocatore controlla un personaggio che si muove liberamente in un'arena di gioco con visuale dall'alto.
- Il sistema deve generare ondate continue di nemici attorno al giocatore, con difficoltà e numero progressivamente crescenti.
- Il giocatore dispone di almeno un'arma per danneggiare i nemici.
- I nemici devono seguire il giocatore e infliggergli danno riducendo i suoi punti vita.
- Il giocatore possiede una barra della vita che rappresenta i punti vita disponibili. Quando i punti vita raggiungono zero (morte del giocatore), la partita termina.
- Il sistema deve mostrare una schermata di game over alla morte del giocatore.
- Ogni nemico eliminato fornisce punti esperienza al giocatore.
- Al raggiungimento di una determinata quantità di esperienza, il giocatore deve poter scegliere un upgrade tra quelli disponibili.
- Il sistema deve permettere al giocatore di ricominciare la partita dalla schermata di game over.

Requisiti non funzionali

- Il gioco deve essere eseguibile sui principali sistemi operativi moderni (Windows, macOS, Linux).

¹ https://store.steampowered.com/app/1794680/Vampire_Survivors/

- La generazione e gestione delle ondate di nemici deve avvenire senza rallentamenti percepibili.
- Il sistema deve mantenere tempi di risposta immediati ai comandi del giocatore (input lag minimo).

1.2 Modello del Dominio

Il dominio del gioco è composto da un insieme di entità che interagiscono fra loro all'interno di un'arena di gioco. Il sistema rappresenta una partita in tempo reale in cui un giocatore controlla un personaggio che deve sopravvivere a ondate continue di nemici, sfruttando armi e raccogliendo risorse.

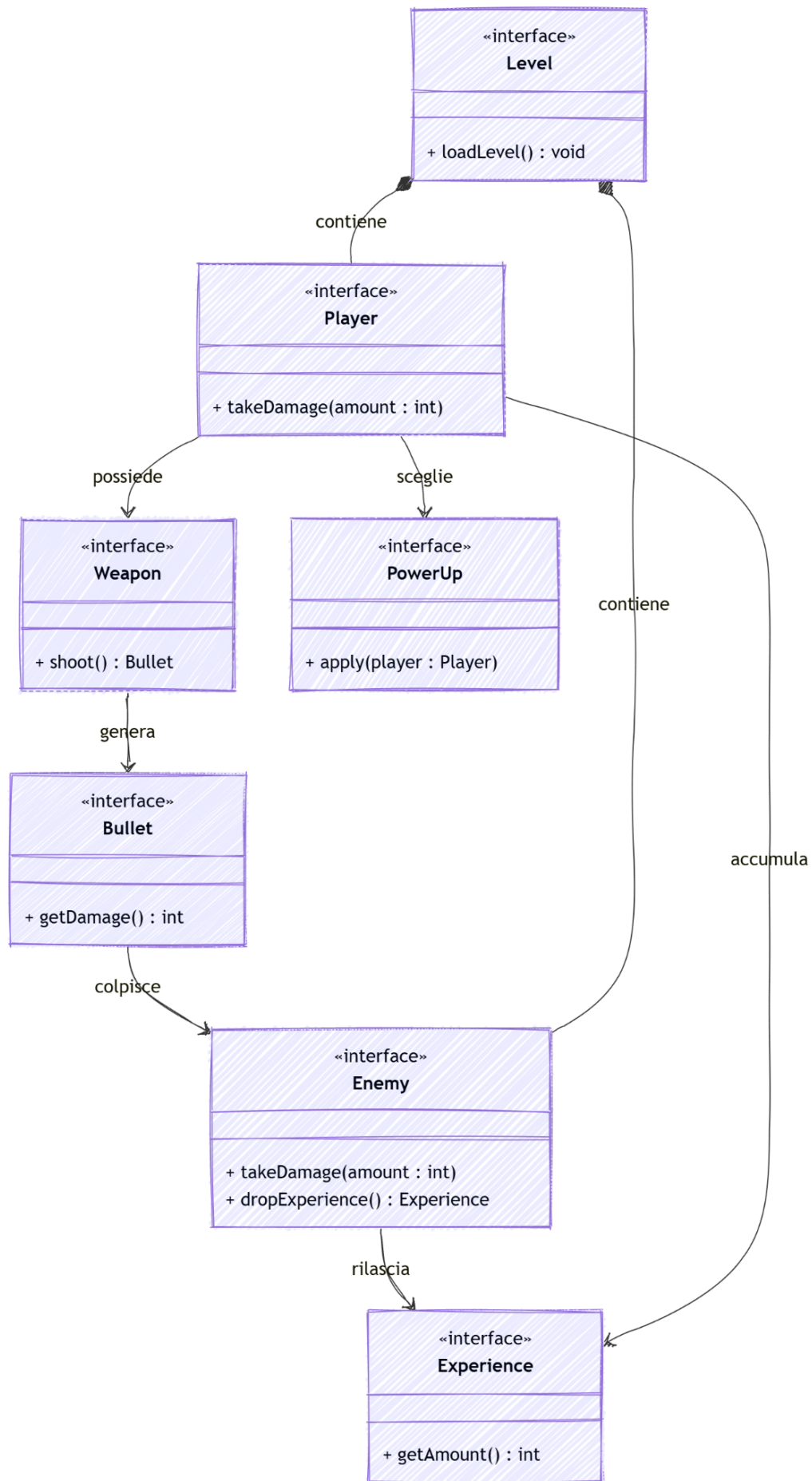
Il giocatore (Player) è l'entità controllata dall'utente. Il personaggio possiede un arma (Weapon) che serve per attaccare i nemici.

I nemici (Enemy) sono entità ostili generate all'interno dell'arena. Essi si muovono verso il giocatore con l'obiettivo di infliggergli danni. I nemici possono essere eliminati tramite gli attacchi generati dalle armi del giocatore. Una volta sconfitti, i nemici rilasciano esperienza, che viene accumulata dal giocatore.

L'arena (Level) è lo spazio di gioco dove si svolge la partita. Essa contiene nemici, munizioni e il giocatore.

L'esperienza (Experience) è una risorsa ottenuta dal giocatore dopo aver sconfitto un nemico. Al raggiungimento di determinate soglie, il personaggio potrà scegliere dei potenziamenti (PowerUp).

I potenziamenti (PowerUp) sono dei miglioramenti applicabili al personaggio, modificandone il comportamento o aumentandone l'efficacia.

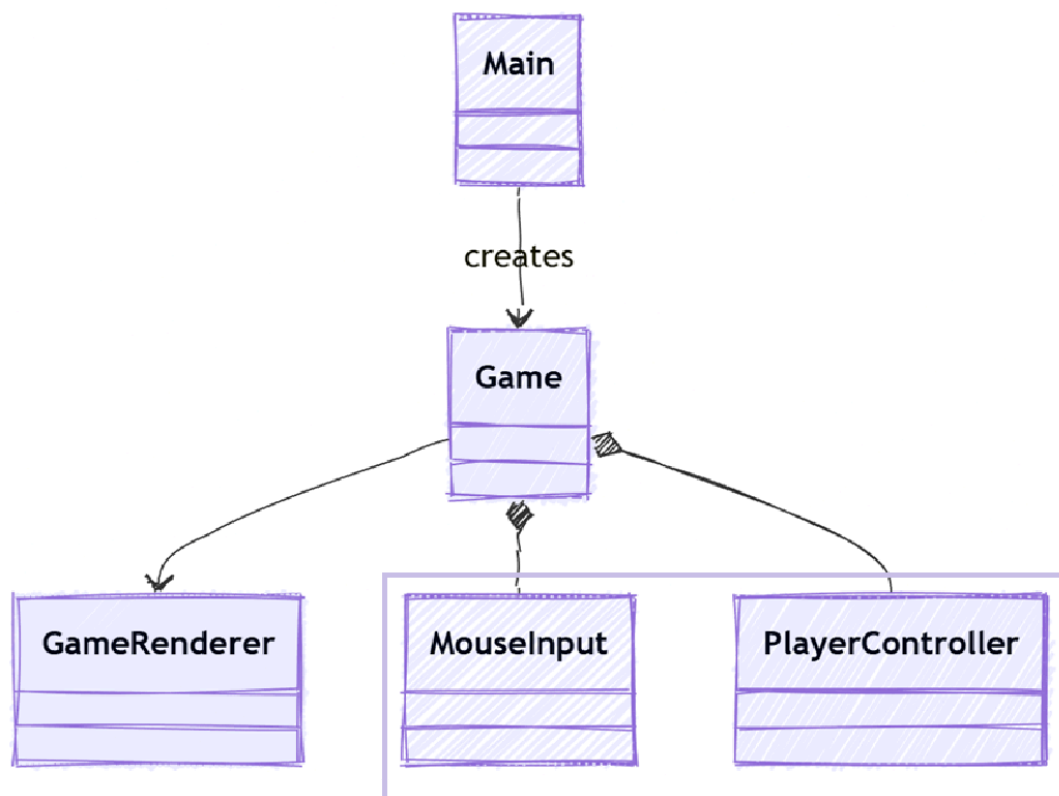


Design

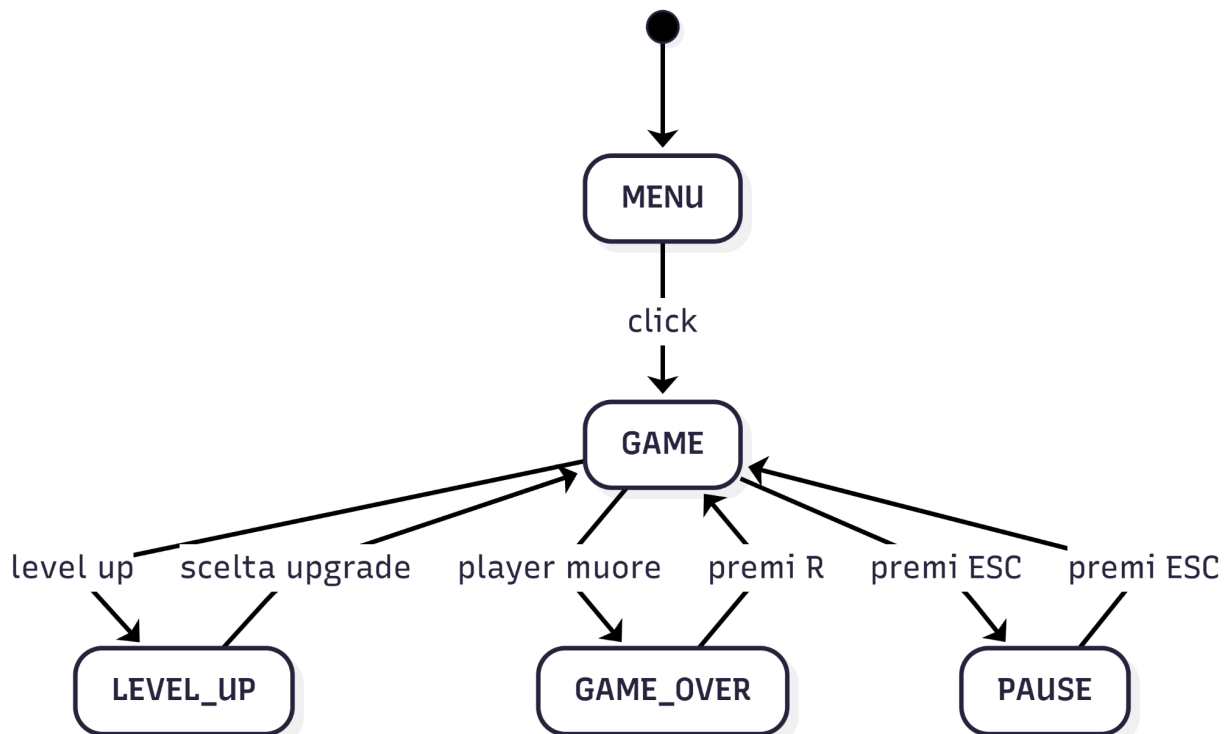
Architettura

L'architettura di shOOt segue il pattern architetturale MVC (Model-View-Controller), che consente di separare chiaramente la logica applicativa, la rappresentazione grafica e la gestione degli input dell'utente.

- Il Model raccoglie tutte le componenti responsabili dello stato del gioco e delle sue regole: la gestione delle entità presenti nel mondo di gioco, i sistemi di progressione, le meccaniche di spawning dei nemici e, più in generale, tutta la logica che determina l'evoluzione della partita. Questa parte dell'applicazione è indipendente dalla grafica e dagli input, rendendo più semplice modificare o estendere le meccaniche di gioco.
- La View è incaricata esclusivamente della rappresentazione visiva dello stato corrente. Essa si occupa del rendering degli elementi di gioco, dell'interfaccia utente e della gestione del punto di vista sul mondo virtuale, senza contenere alcuna logica relativa alle regole del gioco.
- Il Controller ha il compito di intercettare le azioni dell'utente e tradurle in operazioni sul modello. Gli input provenienti da mouse e tastiera vengono quindi elaborati e convertiti in comandi che modificano lo stato del gioco, mantenendo separata la logica di interazione da quella applicativa.

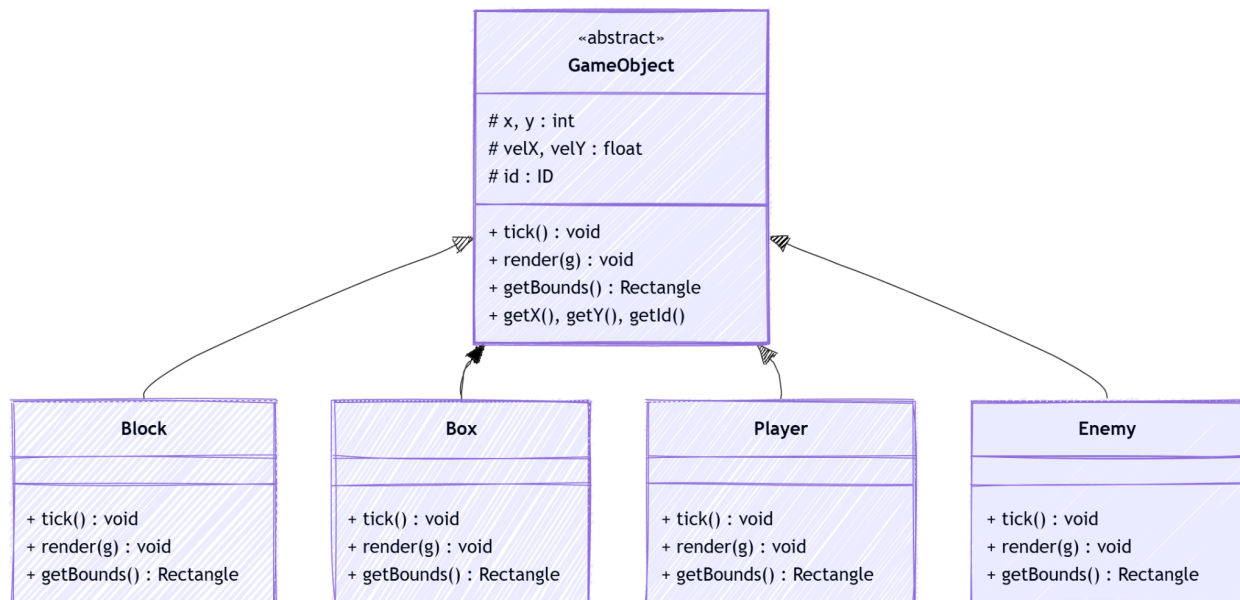


Game state



Il gioco può trovarsi in uno di cinque stati. Si parte dal **MENU**, dove il gioco è in attesa e la logica è inattiva. Un click dell'utente avvia la partita portando allo stato **GAME**, dove tick e render sono entrambi attivi. Se il giocatore accumula abbastanza esperienza si passa a **LEVEL_UP**, dove il mondo si congela e appare il menu di scelta upgrade; scelto un potenziamento si torna a **GAME**. Se invece il giocatore muore si entra in **GAME_OVER**, dove tutta la fisica si ferma e viene mostrata la schermata finale; premendo R la partita ricomincia da capo tornando a **GAME**. Durante lo stato **GAME** il giocatore può premere il tasto di pausa, ed entrare nello stato **PAUSE** dove tutta la logica di gioco è ferma, per poi farla ripartire premendo nuovamente lo stesso tasto, ritornando allo stato **GAME**.

Gerarchia delle entità

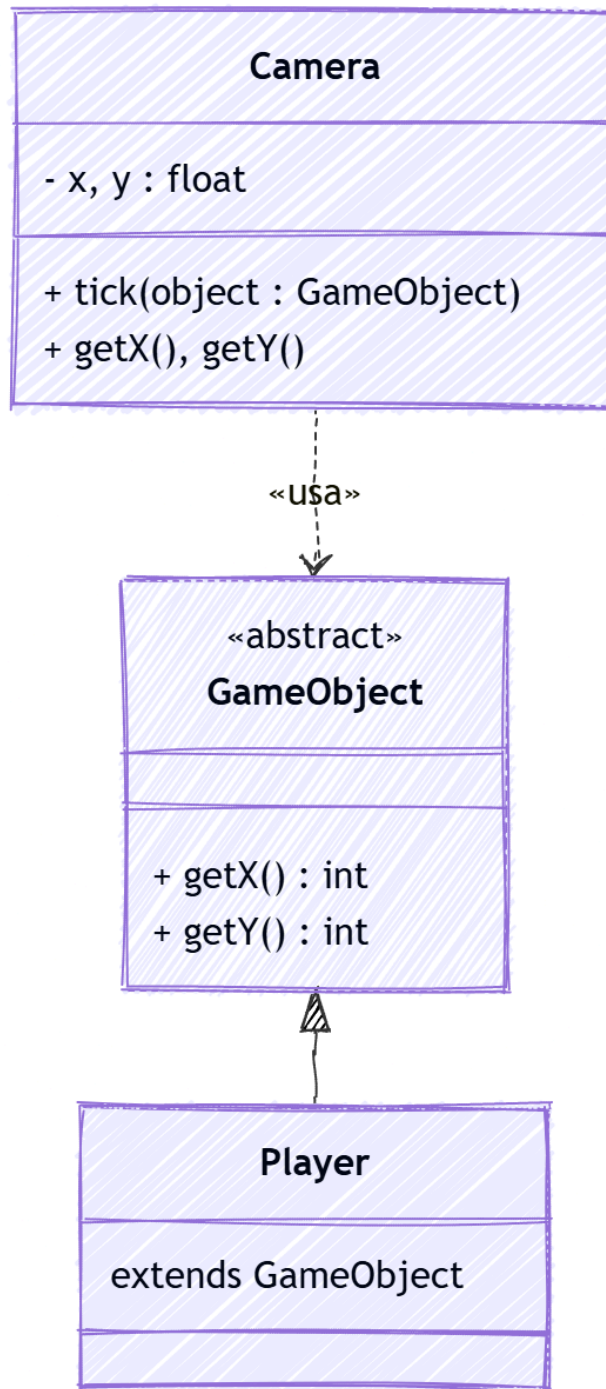


GameObject è la classe astratta da cui derivano tutti gli oggetti presenti nel gioco. Definisce le proprietà comuni a qualsiasi entità (posizione, velocità, identificatore), e impone a ogni sottoclasse di implementare tre metodi: `tick` per aggiornare il proprio stato ad ogni frame, `render` per disegnarsi sullo schermo, e `getBounds` per fornire il proprio rettangolo di collisione. **Block**, **Box**, **Player** e **Enemy** estendono **GameObject** e forniscono ciascuno la propria implementazione concreta di questi metodi, con comportamenti completamente diversi fra loro.

Design dettagliato

Vera Murgia

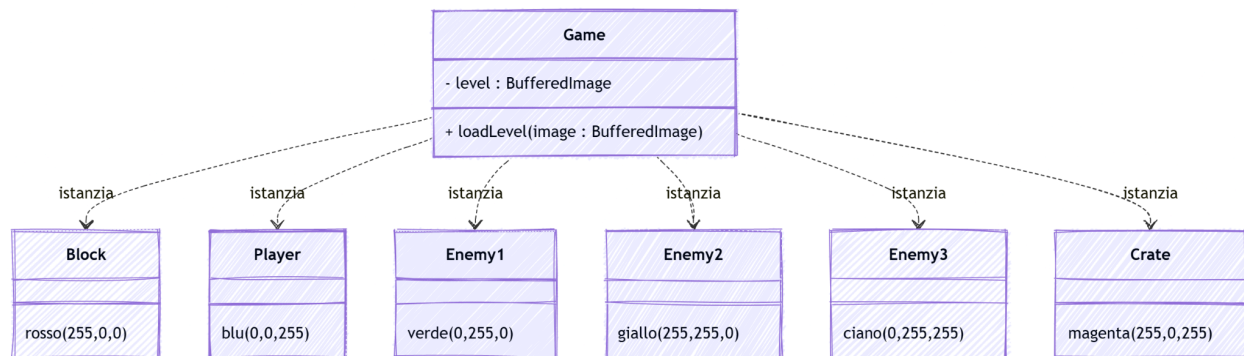
1. Disaccoppiamento della Camera dalla logica di gioco
 - Problema: La camera deve seguire il giocatore durante la partita, ma il sistema di rendering non deve dipendere direttamente dalla classe **Player**, un accoppiamento che renderebbe difficile in futuro seguire altri oggetti o cambiare il soggetto del focus.
 - Soluzione: La camera segue automaticamente il giocatore durante la partita, tenendolo sempre al centro dello schermo. Per fare questo, ad ogni aggiornamento legge la posizione del soggetto che deve seguire e si sposta di conseguenza, evitando però di andare oltre i bordi della mappa. La scelta progettuale rilevante è che la camera non è legata specificamente al giocatore: può seguire qualsiasi oggetto di gioco, rendendo semplice in futuro cambiare il soggetto del focus senza dover modificare il suo funzionamento



2. Caricamento della mappa

- Problema: La mappa di gioco deve essere definita in modo semplice e modificabile, specificando dove posizionare blocchi, giocatore, nemici e oggetti.
- Soluzione: La mappa è codificata come un'immagine PNG, dove ogni pixel rappresenta una cella del mondo. L'immagine è letta pixel per pixel e interpreta il colore RGB come tipo di oggetto da istanziare: rosso puro (255,0,0) per i blocchi, blu puro (0,0,255) per il giocatore, verde puro (0,255,0) per Enemy1, giallo

(255,255,0) per Enemy2, ciano (0,255,255) per Enemy3, e magenta (255,0,255) per le casse. Questo approccio permette di disegnare e modificare le mappe con qualsiasi editor grafico, senza toccare il codice.

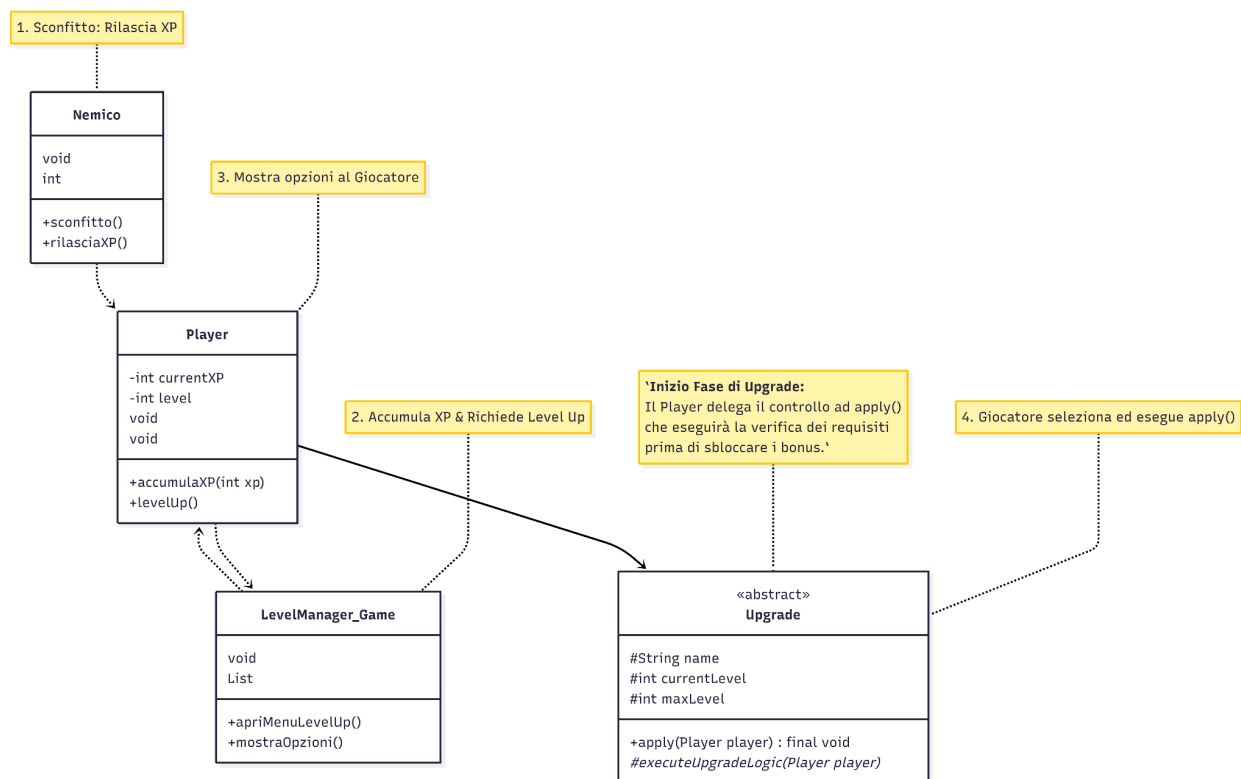


Peter Ibrahim

Raccolta dell'Esperienza e Gestione dei Potenzamenti (Upgrade)

Problema: Man mano che il giocatore sconfigge i nemici, accumula punti esperienza (XP) per salire di livello. Al momento del passaggio di livello, il gioco deve offrire la possibilità di scegliere tra diversi potenziamenti (come velocità, danni, salute ed evasione). Se ogni singolo potenziamento gestisce in autonomia i controlli di sicurezza, ci sarebbe il forte rischio che lo sviluppatore duplica il codice di convalida o si dimentichi di bloccare il potenziamento una volta raggiunto il livello massimo, permettendo al giocatore di sbloccare bonus infiniti e rompere il bilanciamento del gioco.

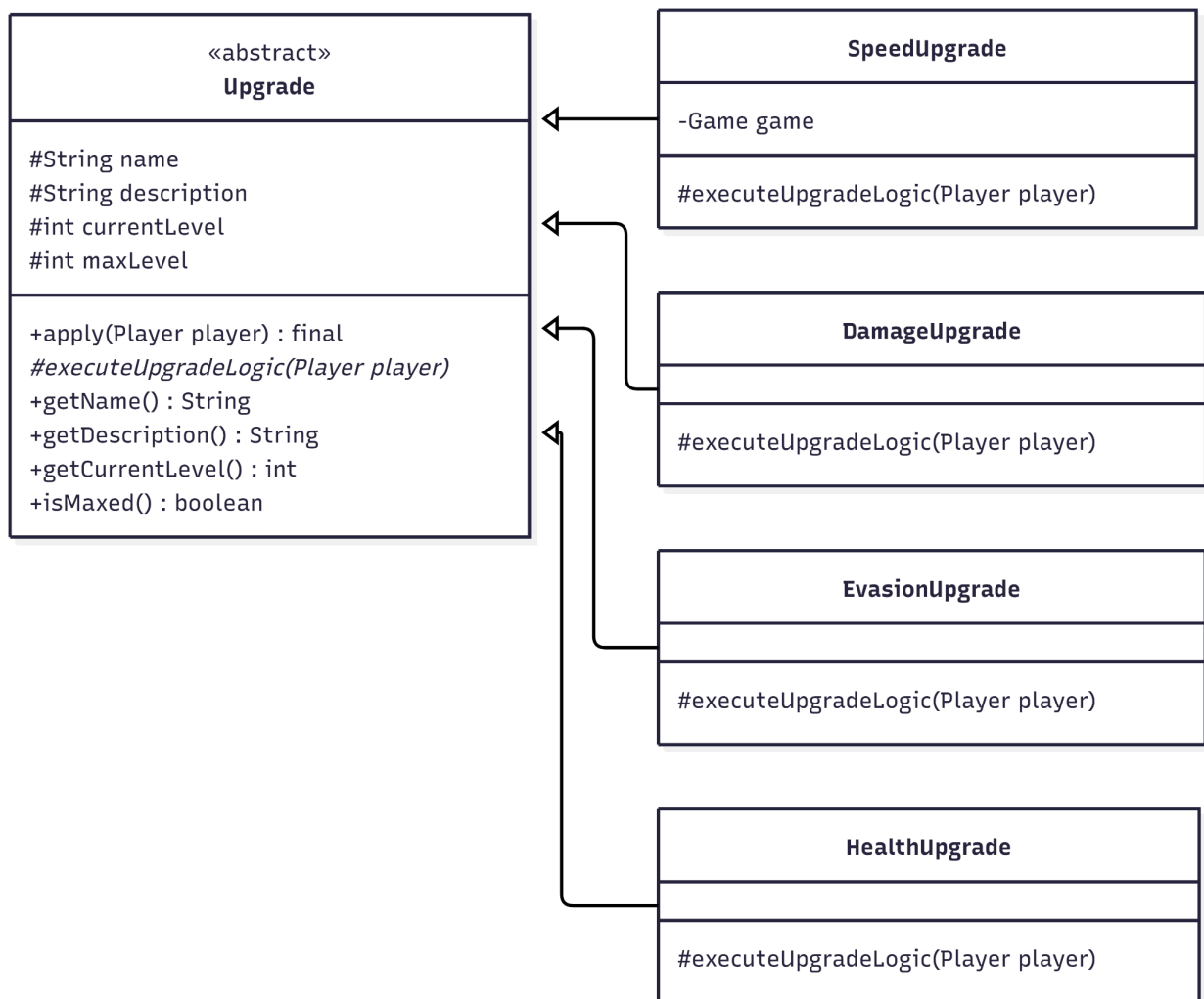
Soluzione: Il sistema risolve il problema applicando il Design Pattern Template Method all'interno della classe astratta Upgrade. Il metodo principale `apply(Player player)` è dichiarato `final`, impedendo alle sottoclassi di modificarne la struttura. Questo metodo centralizza i controlli di sicurezza: verifica prima se il potenziamento è già al massimo livello (`isMaxed()`) e, solo in caso negativo, esegue l'effetto specifico tramite il metodo astratto `executeUpgradeLogic(player)`, incrementando automaticamente e in modo sicuro il contatore del livello (`currentLevel++`). In questo modo, l'aggiunta di nuovi potenziamenti derivati dalla raccolta di XP diventa un'operazione sicura, priva di ridondanze e immune da bug legati al superamento dei limiti di livello.



Gestione ed estensione dei potenziamenti (Upgrade)

Problema: Quando si creano diversi tipi di potenziamenti per il giocatore (velocità, danni, munizioni), c'è il rischio di duplicare i controlli di sicurezza (es. verificare se si è già raggiunto il livello massimo) o di dimenticarsi di aggiornare i contatori dei livelli, introducendo bug o ridondanza nel codice.

Soluzione: La classe Upgrade applica il Design Pattern Template Method tramite il metodo apply(). Questo metodo è dichiarato final per impedire alle sottoclassi di modificarlo, e centralizza in modo sicuro il controllo del livello massimo e l'incremento del livello attuale. Le singole classi derivate devono solo preoccuparsi di implementare la propria logica specifica (il "cosa fa" il potenziamento) all'interno del metodo protetto executeUpgradeLogic(), garantendo un'estensione sicura e priva di errori.



Implementazione MVC tramite Wrapper Class (Player.java)

Problema: Il framework di gioco richiedeva entità conformi alla classe astratta `GameObject` per l'inserimento nel ciclo vitale dell'Handler. L'inserimento diretto della logica di dominio, del rendering e dell'I/O all'interno di un'unica classe avrebbe generato un anti-pattern (God Class), rendendo il codice non testabile e strettamente accoppiato alle librerie grafiche AWT.

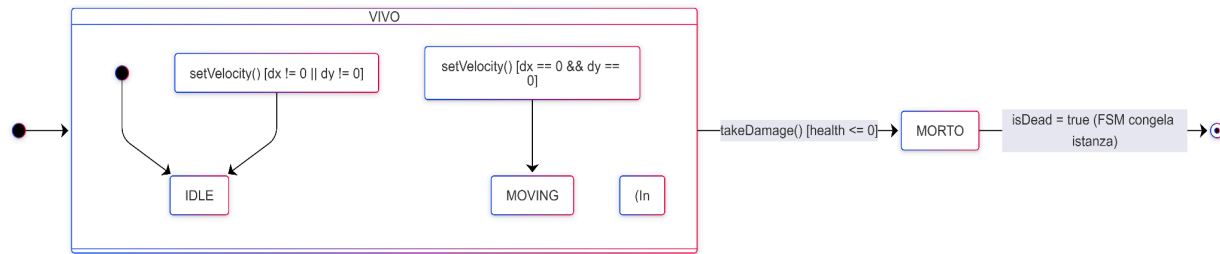
Soluzione: È stata implementata una struttura ad "Aggregatore" (Wrapper). La classe `Player` estende `GameObject` ma funge unicamente da facciata (Facade Pattern semplificato), delegando le responsabilità ai sottomoduli del pattern MVC: `PlayerModel` (fisica e telemetria), `PlayerView` (rendering dello SpriteSheet) e `PlayerController` (lettura hardware). Nel metodo `tick()`, il Wrapper orchestra l'aggiornamento sincronizzando le coordinate assolute del motore grafico con quelle calcolate internamente dal Modello.



Gestione del Ciclo di Esecuzione (FSM)

Problema: Un engine privo di gestione logica degli stati esegue iterazioni continue di calcolo delle coordinate (fisica) e di rilevamento collisioni anche in assenza di entità attive, sprecando cicli di elaborazione e rischiando dereferenziazioni di puntatori nulli.

Soluzione: Il flusso del ciclo vitale dell'entità giocatore (PlayerModel) è stato integrato con un pattern architetturale basato su una Macchina a Stati Finiti (FSM). Questo design funge da barriera logica: quando il flag interno `isDead()` restituisce `true`, le interazioni fisiche e i calcoli vettoriali del personaggio vengono sospesi. Questo previene ricalcoli matematici superflui (come l'accumulo di danni post-mortem) e isola in modo netto lo stato della logica da quello del rendering grafico gestito dalla View.



Risoluzione dell'Input Lag e Movimento Vettoriale

Problema: L'intercettazione standard degli eventi hardware (KeyListener) genera un movimento "scattoso" causato dal delay di ripetizione del sistema operativo, rendendo impraticabile un movimento fluido e responsivo in diagonale.

Soluzione: Il PlayerController è stato ingegnerizzato abbandonando il paradigma event-driven puro a favore di un sistema a Polling Bufferizzato. La pressione e il rilascio dei tasti aggiornano dinamicamente un HashSet (Collezione Set senza duplicati). Ad ogni frame, il metodo `update()` interroga l'insieme dei tasti attivi per comporre un vettore di direzione risultante (dx, dy), che viene successivamente iniettato nel PlayerModel garantendo spostamenti omnidirezionali fluidi a prescindere dal framerate.

Puntamento Vettoriale e Traslazione Spaziale

La gestione dell'input offensivo (MouseInput) agisce da ponte tra il controller hardware e il dominio fisico. La criticità affrontata è stata la discrepanza tra il sistema di riferimento dello schermo (Screen Space) e quello del livello di gioco (World Space). Il Controller interroga dinamicamente la View (Camera) per calcolare un offset di traslazione: aggiungendo le coordinate assolute della telecamera ai dati grezzi del cursore, il sistema ricava il vettore di impatto esatto per la generazione dei proiettili, subordinando il tutto alla disponibilità di munizioni per evitare il *flooding* dell'Handler.

Collezioni Dinamiche e Telemetria Funzionale

Al fine di monitorare le performance di sopravvivenza del giocatore senza inquinare il Modello con cicli iterativi, è stato implementato un sistema di telemetria all'interno del `PlayerModel`. L'estrazione statistica (es. media e picco dei danni) sfrutta la Java Stream API combinata a Method Reference (`Integer::intValue`). La sicurezza delle operazioni è garantita dal tipo monadico `OptionalDouble`, che fornisce un fallback memory-safe contro potenziali eccezioni di divisione per zero in assenza di storico danni.

Rendering Vettoriale e Disaccoppiamento della Hitbox

Il modulo grafico (`PlayerView`) rispetta rigorosamente il pattern architetturale della `Passive View`. Durante l'integrazione degli asset (`SpriteSheet`), è emersa una discrepanza tra il Bounding Box fisico (fissato a 32x32) e la risoluzione nativa dei frame (32x15). Per mantenere intatta la matematica delle collisioni, si è optato per un Texture Scaling a runtime: il metodo `drawImage` applica uno stretching dell'asset in fase di rendering, garantendo che l'ingombro visivo coincida al 100% con la Hitbox logica

Leroy Fabbri

Gestione del movimento dei nemici

- Problema: I nemici devono seguire il giocatore, non oltrepassando i muri volandoci sopra e non uscire dai limiti della mappa di gioco. Un modello troppo semplice causava errori e l'impossibilità dei nemici di rientrare nella mappa e seguire correttamente il personaggio principale.
- Soluzione: Il movimento del nemico è dettato da una normalizzazione della velocità verso il player (non "raddoppia" la velocità se si muove sia sull'asse x e y contemporaneamente). Viene utilizzato il metodo di risoluzione asse per asse per la collisione con i muri e gli ostacoli: le coordinate vengono salvate su due variabili a parte, e all'ottenimento delle nuove coordinate viene testato se

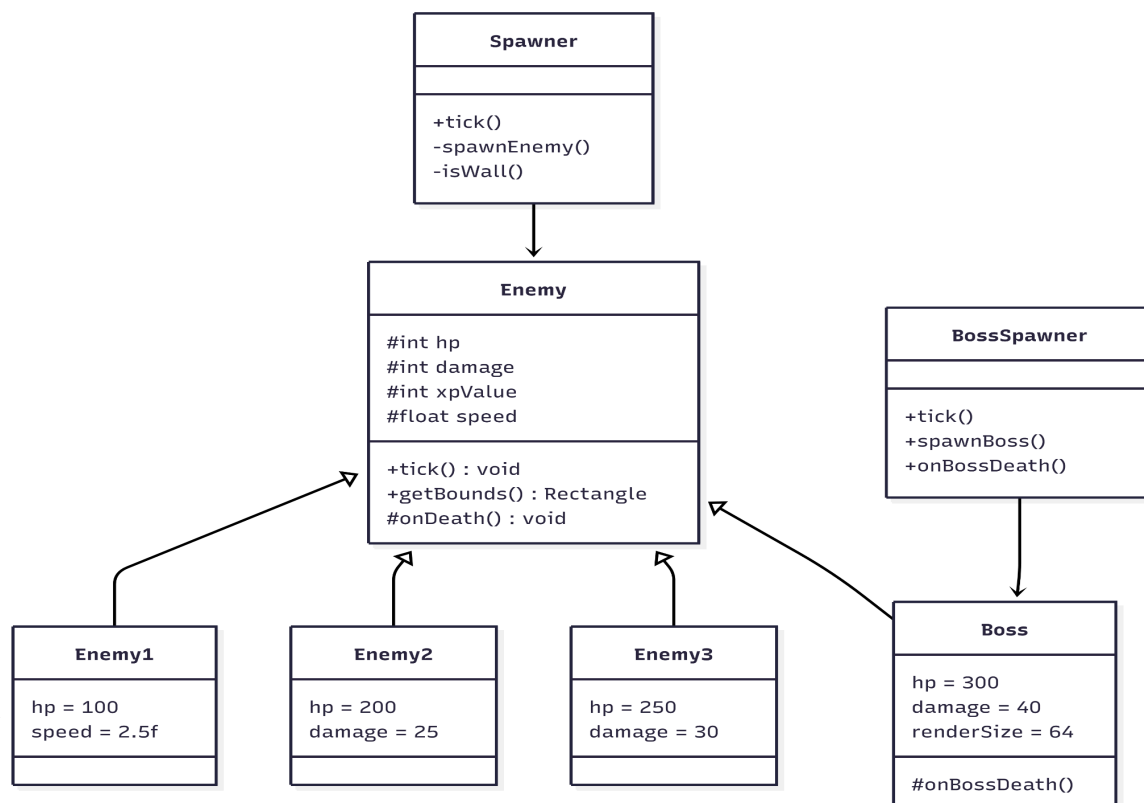
collidono contro un muro, in questo caso viene bloccata la coordinata che ne causa l'intersezione, evitando che il nemico resti bloccato e non si possa più muovere.

Generazione dei nemici

- Problema: il nemico poteva essere generato all'interno di un muro, causandone la rottura nei movimenti e l'impossibilità di essere eliminato.
- Soluzione: prima dello spawn, viene eseguito un try da 30 tentativi, in cui vengono selezionate le coordinate casualmente, al fine di far generare un nemico. Dopo 30 tentativi (molto improbabile ma comunque possibile) il nemico non viene generato, e ricomincia il timer

Sistema Boss

- Problema: il boss deve essere unico e fornire benefici diversi dopo essere stato ucciso
- Soluzione: il boss, un nemico più forte e facilmente riconoscibile (è più grande dei nemici normali), nasce in un punto predeterminato della mappa (più o meno al centro), ha un movimento più lento, danni e vita maggiorata, fornisce molta più esperienza (10 nemici verdi) e lascia a terra una cassa di munizioni nel punto in cui è morto



- **HandlerTest**: verifica l'aggiunta e rimozione di oggetti dalla lista, il corretto funzionamento di `getPlayer()`, e lo svuotamento completo della lista.
- **CameraTest**: verifica il corretto centramento della camera sull'oggetto seguito e il clamping
- **SpriteSheetTest**: verifica l'estrazione corretta di tile validi (con e senza dimensioni personalizzate) e il lancio di `IllegalArgumentException` per coordinate fuori dai limiti.

Peter Ibrahim

- **Damage Upgrade Test**: Verifica dello stato iniziale dell'upgrade e del moltiplicatore del danno del player
- **Max Level Upgrade test** : Verifica il livello massimo del level-up
- **XP Accumulation And Level Up Test** : Verifica delle condizioni di partenza della XP al livello 1 e l'accumulazione dell'XP
- **Get Random Upgrades Filters Maxed** : Controlla l'esistenza delle 4 potenziamenti e l'arrivo del livello massimo.

Kirollos Habashi

Per quanto concerne il dominio fisico e logico del giocatore, la validazione automatizzata è stata condotta tramite suite JUnit 5 (`PlayerModelTest`). Rifiutando un approccio superficiale basato sul "Boilerplate Testing" (verifica sterile di getter e setter), la stesura dei test si è concentrata esclusivamente sul Behavioral Testing, mettendo sotto pressione il modello matematico per scovare vulnerabilità strutturali e limiti di soglia (Edge Cases). Nello specifico, le asserzioni garantiscono:

- **testInitialState**: verifica la corretta inizializzazione dei parametri vitali (salute attuale e massima) e dello stato logico di sopravvivenza al momento dell'istanziamento del Modello.
- **testTakeDamageAndIFrames**: verifica la precisione del decremento vitale e, soprattutto, la tenuta temporale della finestra di invulnerabilità (I-Frames), garantendo che danni massicci consecutivi entro la soglia di un secondo vengano ignorati.
- **testHealLimits**: verifica il blocco matematico superiore della salute (Clamping), assicurando che il recupero dei punti vita non generi mai un superamento della soglia massima consentita (Over-healing).
- **testUpdatePositionVector**: verifica la corretta traslazione spaziale delle coordinate X e Y, garantendo che il vettore di velocità calcolato venga correttamente moltiplicato per la statistica di velocità (speed) e che il flag di movimento (`isMoving`) scatti correttamente.
- **testDodgeMechanic**: verifica l'integrità del filtro stocastico di evasione, dimostrando che un'impostazione forzata al 100% della probabilità di schivata annulli completamente i danni in ingresso.

- **testDeathState:** verifica il corretto scatto della Macchina a Stati al ricevimento di un danno letale, assicurando che la variabile della salute si fermi allo zero assoluto (evitando valori negativi) e che il flag `isDead` si attivi per bloccare il Game Loop.

Leroy Fabbri

- **BulletTest:** verifica che il proiettile si muova e se scompare colpendo un blocco
- **BossSpawnerTest:** verifica che il boss nasca correttamente, che non spawni prima del tempo e che rilasci una cassa alla sua morte.
- **EnemyTest:** Verifica che il nemico si avvicini al giocatore, se scompare dopo essere stato ucciso, che aggiunga XP alla barra dell'XP del giocatore dopo essere stato eliminato, e che non venga rimosso se è ancora in vita.
- **SpawnerTest:** Verifica che non spawni un nemico prima dello scadere del timer, e che spawni correttamente alla fine del timer.

Note di sviluppo

Vera Murgia

- **Utilizzo della libreria Java Swing:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/view/Window.java#L22>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/view/GamePanel.java#L12>
- **Utilizzo della libreria Java AWT:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/view/Window.java#L24>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/view/GamePanel.java#L16>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Handler.java#L34>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Game.java#L26>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/loader/SpriteSheet.java#L3>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/loader/ResourceLoader.java#L3>

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/loader/BufferedImageLoader.java#L3>
- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/GameObjects/GameObject.java#L3>
- **Utilizzo di BufferStrategy:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/view/GameRenderer.java#L109>
- **Utilizzo di BufferedImage:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/loader/SpriteSheet.java#L3>
- **Utilizzo di multithreading tramite Runnable:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Game.java#L26>
- **Utilizzo di Lambda:**
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Handler.java#L25>
 - <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Handler.java#L34>

Durante la fase iniziale di sviluppo del progetto abbiamo consultato alcune risorse online, in particolare una playlist su YouTube dedicata allo sviluppo di un videogioco in Java:

- [Java Game Programming Wizard Top Down Shooter](#)

Il ciclo di gioco (game loop) è ispirato al modello comunemente attribuito a Notch (Markus Persson) per il suo videogioco Minecraft². Questo ciclo di gioco si basa su un aggiornamento ciclico separato tra tick e rendering.

Questi materiali sono stati utilizzati esclusivamente come supporto per comprendere l'impostazione generale di un progetto object-oriented di questo tipo. Successivamente, le varie parti del progetto sono state sviluppate e adattate dal gruppo in modo progressivo, in base alle esigenze specifiche del sistema.

Peter Ibrahim

- **Utilizzo della libreria Java AWT:**
 - <https://github.com/Veenya/OOP25-shoot/blob/6e46fa7a26b25d852c04357061b249ed5ced4104/src/main/java/it/unibo/shoot/GameObjects/Block.java#L5-L6>

² <https://www.minecraft.net/>

- <https://github.com/Veenya/OOP25-shoot/blob/6e46fa7a26b25d852c04357061b249ed5ced4104/src/main/java/it/unibo/shoot/GameObjects/Block.java#L6>
- <https://github.com/Veenya/OOP25-shoot/blob/6e46fa7a26b25d852c04357061b249ed5ced4104/src/main/java/it/unibo/shoot/GameObjects/Block.java#L7>
-
- **Utilizzo di BufferedImage:**
 - <https://github.com/Veenya/OOP25-shoot/blob/6e46fa7a26b25d852c04357061b249ed5ced4104/src/main/java/it/unibo/shoot/GameObjects/Crate.java#L45>

Kirollos Habashi

Tecnologie e Librerie Utilizzate

Utilizzo delle Java Stream API e Tipi Monadici :

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/PlayerModel.java#L185>
- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/PlayerModel.java#L196>

Utilizzo del Java Collections Framework (Interfaccia Set e HashSet):

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/controller/PlayerController.java#L22>

Utilizzo delle interfacce nativa Java AWT per I/O Hardware:

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/controller/PlayerController.java#L16>
- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/controller/MouseInput.java#L13>

Utilizzo della libreria Java AWT (Geometria 2D):

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/PlayerModel.java#L149>

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/Player.java#L131>

Utilizzo della libreria Math (Numeri Pseudo-casuali):

- <https://github.com/Veenya/OOP25-shoot/blob/b0c308eed020bab652ae127dcbbd7ecbb613308e/src/main/java/it/unibo/shoot/model/PlayerModel.java#L125>

Leroy Fabbri

Utilizzo della libreria Java AWT

- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L155-L183>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/GameObjects/Bullet.java#L64>

Utilizzo di BufferedImage

- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L17>
- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Boss.java#L12>

Utilizzo delle Java Stream API e Tipi Monadici (Optional)

- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L127-L128>
- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L147-L150>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Enemy.java#L75-L76>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/GameObjects/Bullet.java#L46-L49>

Utilizzo di espressioni Lambda

- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L128>

Utilizzo di Random

- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Spawner.java#L54-L55>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Boss.java#L35>

Utilizzo di Math

- <https://github.com/Veenya/OOP25-shoot/blob/859c99393854386f65f705e7bcf85dd2bc8122a2/src/main/java/it/unibo/shoot/model/Enemy.java#L107>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/GameObjects/Bullet.java#L25>

Ereditarietà

- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Boss.java#L12>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Boss.java#L45>
- <https://github.com/Veenya/OOP25-shoot/blob/8664493dd9f4afa9d2010e123991a56c92965989/src/main/java/it/unibo/shoot/model/Enemy.java#L213>

Risorse Esterne e Ispirazioni Architetture: Durante la fase iniziale di sviluppo del progetto abbiamo consultato alcune risorse online, in particolare una playlist su YouTube dedicata allo sviluppo di un videogioco in Java: *Java Game Programming Wizard Top Down Shooter*.

Commenti finali

Autovalutazione e lavori futuri

Vera Murgia

Il mio contributo al progetto si è concentrato principalmente nell'organizzazione del lavoro, e sul coordinamento delle attività di gruppo. Ho cercato di distribuire i compiti tra i membri, stabilire scadenze interne, modalità organizzative, e mantenere una comunicazione costante. In diversi momenti ho assunto il ruolo di guida, sollecitando anche il completamento di parti mancanti per evitare ritardi.

Un punto di forza del mio lavoro è stata la gestione del tempo, provando sempre prontamente a rispettare le scadenze. Questo mi ha permesso di dividere il carico di lavoro in modo uniforme nel tempo. Tra i punti di debolezza, riconosco che sono sensibile alla pressione delle scadenze e non lavoro bene in situazioni di stress. Inoltre in alcune fasi avrei potuto dedicare più tempo e attenzione alla revisione del codice, soprattutto riguardo alle specifiche richieste dagli esaminatori.

Per quanto riguarda le esperienze precedenti, avevo già iniziato un progetto simile con un gruppo differente un'anno prima, che però non è stato portato a termine. Questa esperienza mi ha comunque aiutato a capire meglio le difficoltà legate al lavoro di gruppo.

Peter Ibrahim

Lo sviluppo del sistema di Level-up e dei relativi modificatori di stato ha permesso di raggiungere un grado di estensibilità soddisfacente, sfruttando il polimorfismo tramite la classe astratta (o interfaccia) Upgrade da cui ereditano le implementazioni specifiche (Damage, Speed, Health, Evasion). Questo approccio isola la logica di calcolo del bonus dal PlayerModel, garantendo l'integrità del ciclo di vita dei dati e rispettando i vincoli architetturali MVC. Tuttavia, l'attuale meccanismo di trigger del level-up presenta delle limitazioni strutturali.

L'assegnazione e l'applicazione dei potenziamenti sono attualmente gestite in modo diretto e sincrono al raggiungimento di una determinata soglia di esperienza, il che vincola l'architettura a una progressione rigidamente lineare. Sebbene computazionalmente immediato ed economico, questo approccio risulta limitato nel *Game Feel*, poiché l'applicazione istantanea interrompe il flusso d'azione senza offrire una transizione fluida o una vera interfaccia di scelta dinamica (UI di selezione dell'upgrade) integrata nel loop di gioco

Lavori futuri: Un'evoluzione critica del sistema richiederà il deprecaimento dell'applicazione sincrona dei bonus, migrando verso un'architettura basata sul Pattern Strategy combinata con un sistema a eventi (Observer Pattern). Ciò consentirà di gestire code di modificatori temporanei o condizionali (es. perk attivi solo a salute bassa) e di implementare una schermata

di pausa interattiva per la selezione degli upgrade senza spezzare il determinismo del ciclo di aggiornamento logico."

Kirollos Habashi

Il refactoring del PlayerModel ha permesso di raggiungere un grado di robustezza soddisfacente, isolando matematicamente la logica di dominio dal rendering visivo e garantendo l'integrità del ciclo vitale tramite l'architettura MVC e il disaccoppiamento temporale. Tuttavia, l'attuale motore fisico presenta delle limitazioni geometriche.

Il rilevamento delle collisioni è attualmente delegato alla classe nativa `java.awt.Rectangle`, il che vincola l'architettura a un rilevamento di tipo AABB (Axis-Aligned Bounding Box). Sebbene computazionalmente economico, questo approccio risulta "spigoloso" nel Game Feel, causando talvolta compenetrazioni non volute o hitbox percepite come ingiuste dal giocatore durante i movimenti diagonali.

Lavori futuri: Un'evoluzione critica del motore richiederà il deprecaimento dell'interfaccia AWT per la fisica, migrando verso un sistema a hitbox composite (es. bounding box circolari) e implementando il Teorema degli Assi Separatori (SAT) per una risoluzione vettoriale precisa degli impatti e dello scivolamento sui bordi della mappa.

Leroy Fabbri

Lo sviluppo del sistema dei nemici permette di ottenere un'ampia estendibilità grazie all'ereditarietà tra `Enemy`, `Enemy1`, `Enemy2`, `Enemy3`, e `Boss`, dove le sottoclassi modificano solamente i vari parametri come il danno, la vita e l'xp ottenuta, in aggiunta all'Override di `Boss` che permette funzioni aggiuntive, come lo spawn della cassa. Questo isola le funzioni in comune, come il movimento e le collisioni, che evita le duplicazioni e la ridondanza di codice. Tuttavia il meccanismo di spawn e di movimento dei nemici non è ottimale. Attualmente i nemici vengono spawnati con cadenza fissa e una selezione di nemici omogenea, ma che non offre una varietà in base al livello del giocatore, e non aumenta la frequenza dei nemici con il passare del tempo, e sembrano poco intelligenti, incastrandosi negli angoli dei muri, creando una mancanza artificiale di nemici.

Queste mancanze possono essere migliorate con un sistema di generazione di nemici a ondate, che possono permettere la nascita di un solo tipo di nemico o una generazione più accelerata con lo scorrere dei secondi e con i nemici eliminati, e a un miglior sistema di IA del nemico che evita gli ostacoli tornando indietro e percorrendo strade meno dirette.

Guida utente

- La prima schermata che compare è la schermata di menu, per avviare la partita si deve cliccare lo schermo con il mouse



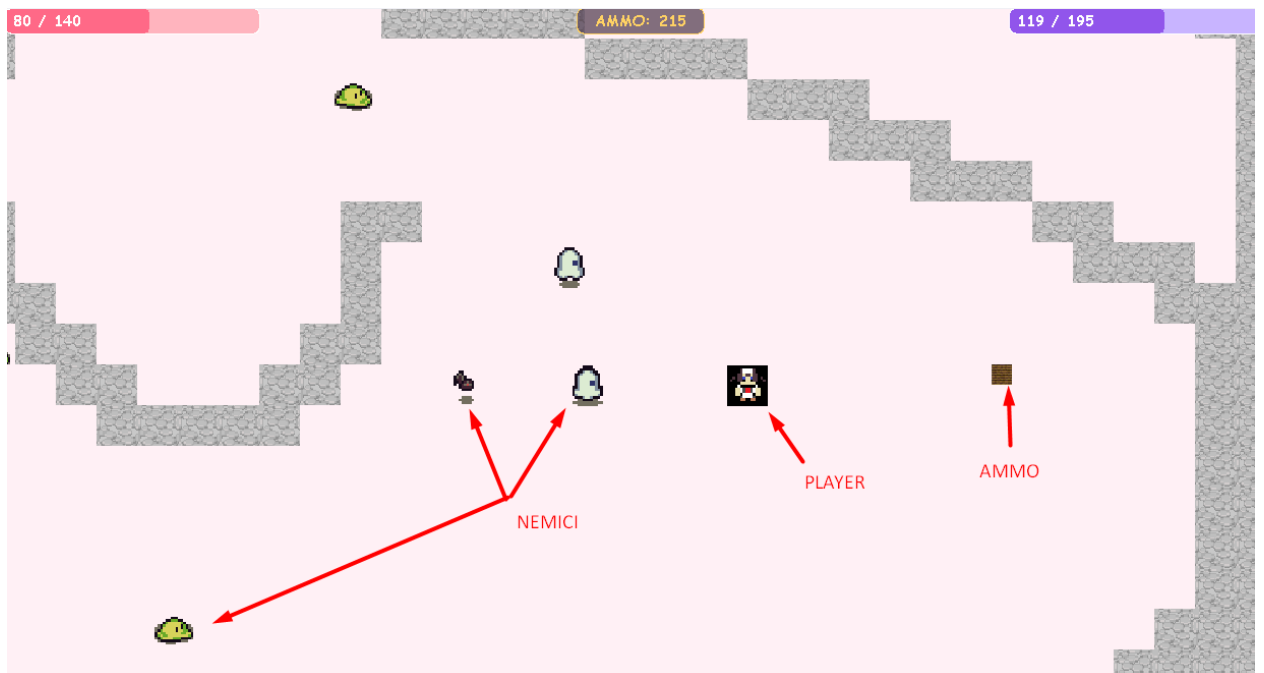
- All'avvio della partita ci si ritrova all'interno dell'arena.
- Per muoversi si utilizzano i tasti W (su), A (sinistra), S (giù), D (destra).
- In alto a sinistra si trova la barra della vita, che parte verde (piena), e quando i nemici entrano in contatto con il giocatore diventa rossa, indicando i punti di vita persi. Quando la barra è totalmente rossa la partita si conclude.
- In alto a destra si trova la barra dell'esperienza, che parte rossa (vuota), e si riempie di blu quando si uccidono i nemici. Quando la barra è completamente blu apparirà un

menù con gli upgrade disponibili, che si selezionano cliccandoci sopra. Una volta selezionato l'upgrade la barra tornerà vuota, e ciclicamente si potranno aumentare gli upgrade.

- In alto al centro si vede il numero di munizioni (AMMO) che il giocatore ha a disposizione. Le munizioni si usano cliccando col mouse sul punto in cui si vuole sparare.



- Il numero di munizioni è limitato, ma è possibile raccoglierne altre perché presenti all'interno dell'arena.



- I nemici moriranno dopo essere stati colpiti con un numero variabile di proiettili.

- Durante la partita si può entrare in Pausa premendo ESC, e uscire dalla pausa per tornare al gioco premendo nuovamente ESC.



- A conclusione della partita si sceglie se ricominciare (premendo R) o chiudere il gioco (premendo X).

